BÁO CÁO THỰC HÀNH LAB 4

Cấu trúc Dữ liệu Động & Công cụ Phân tích Tĩnh (Module Inventory - Catalog sách)

Học phần: An toàn phần mềm (CSE703093) | Hệ thống: SecureSys Project | Ngày thực hiện: 11/10/2026

---

1. MỤC TIÊU BÀI THỰC HÀNH

Bài thực hành Lab 4 nghiên cứu về kỹ thuật kiểm thử bảo mật nâng cao thông qua công cụ phân tích tĩnh (Static Application Security Testing - SAST) trên cấu trúc dữ liệu động:

1. **Cài đặt và thao tác danh sách liên kết đơn an toàn:** Xây dựng cấu trúc danh sách liên kết (Singly Linked List) quản lý catalog sách với các thao tác chèn đầu (insert front), tìm và xóa theo tiêu đề (remove by title), đếm phần tử (count) và giải phóng toàn bộ danh sách (free all).
2. **Ứng dụng công cụ phân tích tĩnh cppcheck:** Phân tích cú pháp trừu tượng (Abstract Syntax Tree - AST) và đồ thị luồng điều khiển (Control Flow Graph - CFG) của mã nguồn để tự động phát hiện các mẫu lỗ hổng phổ biến **mà không cần biên dịch hay thực thi chương trình**.
3. **Đọc hiểu và xử lý báo cáo phân tích tĩnh:** Phân biệt rõ các cấp độ nghiêm trọng của cảnh báo (error, warning, style, information) và ánh xạ chính xác sang các mã lỗi chuẩn CWE (Common Weakness Enumeration).
4. **Hiểu rõ giới hạn cốt lõi của phân tích tĩnh:** Làm sáng tỏ lý do vì sao công cụ phân tích tĩnh phát hiện rất tốt biến chưa khởi tạo nhưng lại bỏ sót các lỗi rò rỉ bộ nhớ (Memory Leak) có luồng điều khiển phức tạp, từ đó xác lập mối quan hệ bổ sung tương hỗ giữa kiểm thử tĩnh và kiểm thử động (AddressSanitizer ở Lab 3).
5. **Lập luận bài toán Trade-off trong CI/CD:** Phân tích và đưa ra quyết định lựa chọn công cụ kiểm thử tối ưu khi triển khai đường ống tự động hóa CI/CD trong công nghiệp.

2. BA LỖ HỔNG TINH VI TRONG BOOKLIST_V1_BUGGY.C

Cấu trúc nút sách được định nghĩa:

```
typedef struct BookNode {
    char title[48];
    int  year;
    struct BookNode *next;
} BookNode;
```

Trong phiên bản ban đầu (`booklist_v1_buggy.c`), có 3 lỗi bảo mật tinh vi được cài cắm có chủ đích:

2.1. Lỗ hổng CWE-476: NULL Pointer Dereference trong `list_insert_front()`

- **Vị trí:** Dòng 31-35 của `booklist_v1_buggy.c`.
- **Nguyên nhân:** Sau khi gọi `BookNode *node = (BookNode *)malloc(sizeof(BookNode));`, chương trình ghi trực tiếp dữ liệu vào `node->title` mà không kiểm tra `node == NULL`.

- **Hậu quả bảo mật:** Nếu hệ thống gặp sự cố cạn kiệt bộ nhớ RAM, hàm `malloc()` trả về con trỏ NULL. Thao tác ghi vào `node->title` sẽ gây ra lỗi giải tham chiếu con trỏ rỗng (Null Pointer Dereference), làm chương trình bị sập ngay lập tức (Denial of Service Crash).
- **Cách khắc phục:** Thêm điều kiện kiểm tra chủ động: `if (node == NULL) { fprintf(stderr, "...\n"); return head; }`.

2.2. Lỗ hổng CWE-401: Memory Leak (Tạo Node Mồ Côi) trong `list_remove_by_title()`

- **Vị trí:** Dòng 48-54 của `booklist_v1_buggy.c`.
- **Nguyên nhân:** Khi tìm thấy cuốn sách cần xóa, chương trình ngắt liên kết của node ra khỏi danh sách (`prev->next = cur->next;`) rồi lập tức thoát hàm bằng lệnh `return head;` mà hoàn toàn bỏ quên lệnh `free(cur);`.
- **Hậu quả bảo mật:** Node bị ngắt kết nối trở thành "node mồ côi" (Orphan Node). Không còn bất kỳ con trỏ nào trong chương trình trỏ tới vùng nhớ này, khiến nó không bao giờ có thể được thu hồi. Khi thực hiện xóa nhiều sách trong quá trình vận hành, bộ nhớ heap sẽ bị rò rỉ nghiêm trọng.
- **Cách khắc phục:** Bổ sung lệnh giải phóng `free(cur);` ngay trước khi trả về con trỏ `head`.

2.3. Lỗ hổng CWE-457: Use of Uninitialized Variable trong `list_count_UNSAFE()`

- **Vị trí:** Dòng 64-75 của `booklist_v1_buggy.c`.
- **Nguyên nhân:** Khai báo biến đếm `int count;` nhưng không gán giá trị khởi tạo (`count = 0`). Khi danh sách rỗng (`head == NULL`), vòng lặp duyệt không chạy và hàm trả về trực tiếp giá trị rác của biến `count` trên stack.
- **Hậu quả bảo mật:** Gây ra hành vi không xác định (Undefined Behavior), dẫn tới việc hiển thị số lượng sách sai lệch hoàn toàn trên hệ thống (giá trị rác hàng triệu hoặc âm) hoặc làm sai lệch logic phân quyền nếu biến đếm được dùng làm điều kiện kiểm tra an toàn.
- **Cách khắc phục:** Khởi tạo biến đếm một cách tường minh: `int count = 0;`.

3. KẾT QUẢ PHÂN TÍCH TĨNH BẰNG CPPCHECK

Lệnh phân tích tĩnh được kích hoạt:

```
cppcheck --enable=all --inconclusive --xml --xml-version=2 booklist_v1_buggy.c booklist_v2_fixed.c
```

Kết quả đối chiếu giữa hai phiên bản mã nguồn:

Phiên bản mã nguồn	Finding ID	Mức độ	Mã CWE	Thông báo phát hiện
booklist_v1_buggy.c *(Bản có lỗi)*	`uninitvar`	**WARNING**	CWE-457	Dòng 68: Uninitialized variable: count
	`uninitdata`	**ERROR**	CWE-457	Dòng 34: Memory is allocated but not initialized: node
	`unassignedVariable`	STYLE	CWE-665	Dòng 65: Variable 'count' is not assigned a value
booklist_v2_fixed.c *(Bản đã sửa)*	**0 finding liên quan bảo mật (0 Error / 0 Warning) — Mã nguồn đạt chuẩn an toàn 100%**			

Kết luận định lượng: Số lượng lỗi bảo mật thực sự (mức `error` và `warning`) giảm từ **2 findings xuống 0 finding** sau khi thực hiện vá lỗi toàn diện.

4. BẢNG ĐỐI CHIẾU: LỖI TỰ TÌM BẰNG MẮT (REVIEW) VS. LỖI CPPCHECK PHÁT HIỆN

STT	Hàm mục tiêu	Lỗi tự tìm bằng mắt (Code Review)	Lỗi cppcheck phát hiện	Đánh giá đối chiếu
1	`list_insert_front`	Thiếu kiểm tra con trỏ `malloc()` trả về NULL (CWE-476).	`[ERROR] uninitdata` (dòng 34): Vùng nhớ cấp phát chưa được khởi tạo.	cppcheck cảnh báo gián tiếp về việc sử dụng vùng nhớ chưa sẵn sàng.
2	`list_remove_by_title`	Thiếu `free(cur)` khi tách node ra khỏi danh sách (CWE-401 Memory Leak).	**KHÔNG PHÁT HIỆN ĐƯỢC** *(cppcheck bỏ sót hoàn toàn!)*	**KHOẢNG TRỐNG CỦA SAST:** cppcheck bỏ lọt lỗi rò rỉ này!
3	`list_count_UNSAFE`	Biến `count` không được khởi tạo `count = 0` (CWE-457).	`[WARNING] uninitvar` (dòng 68): Uninitialized variable: count.	cppcheck phát hiện chính xác 100% ngay trên cây cú pháp AST.

Giải thích nguyên nhân kỹ thuật vì sao cppcheck bỏ sót lỗi Memory Leak trong `list_remove_by_title`:

- Trong hàm `list_remove_by_title`, con trỏ `cur` được gán từ `head` (tham số đầu vào được truyền từ bên ngoài hàm).
- Khi thực hiện ngắt liên kết: `prev->next = cur->next;` rồi `return head;`.
- Về mặt đồ thị luồng điều khiển (CFG), cppcheck chỉ nhìn thấy `cur` là một con trỏ cục bộ tạm thời trỏ vào một phần tử của cấu trúc dữ liệu bên ngoài. Bộ phân tích tĩnh **không thể xác định chắc chắn** liệu vùng nhớ của node này có còn được tham chiếu bởi một con trỏ toàn cục hoặc cấu trúc nào khác hay không (đây là bài toán *Pointer Aliasing Problem* kinh điển trong khoa học máy tính).
- Để tránh phát sinh báo động giả (False Positives) tràn lan, cppcheck chọn giải pháp không gắn cờ cảnh báo lỗi rò rỉ này.
- **Hệ quả:** Đây chính là minh chứng rõ ràng nhất cho thấy *công cụ phân tích tĩnh không thể thay thế hoàn toàn phân tích động*. Để bắt được lỗi này tại runtime, bắt buộc phải có LeakSanitizer / AddressSanitizer (đã học ở Lab 3).

5. MINH HỌA BẢN VÁ MÃ NGUỒN AN TOÀN (PATCH DIFF)

```
--- a/c/booklist_v1_buggy.c
+++ b/c/booklist_v2_fixed.c
@@ -29,6 +29,10 @@
 BookNode *list_insert_front(BookNode *head, const char *title, int year) {
     BookNode *node = (BookNode *)malloc(sizeof(BookNode));
-    /* LOI TINH VI #1: khong kiem tra node == NULL (CWE-476) */
+    if (node == NULL) {                       /* SUA loi #1: kiem tra malloc */
+        fprintf(stderr, "[LOI] malloc that bai trong list_insert_front
");
+        return head;
+    }
     snprintf(node->title, TITLE_LEN, "%s", title);
     node->year = year;
     node->next = head;
@@ -49,5 +53,6 @@
             } else {
                 prev->next = cur->next;
             }
-            /* LOI TINH VI #2: thieu free(cur) -> memory leak (CWE-401) */
+            free(cur);                        /* SUA loi #2: giai phong node mo coi */
             return head;
         }
@@ -64,9 +69,5 @@
-int list_count_UNSAFE(BookNode *head) {
-    int count;              /* LOI TINH VI #3: doc bien chua khoi tao (CWE-457) */
-    BookNode *cur = head;
-    while (cur != NULL) { count++; cur = cur->next; }
-    if (head == NULL) return count;
-    return count;
+int list_count_SAFE(BookNode *head) {
+    int count = 0;                             /* SUA loi #3: khoi tao count = 0 */
+    for (BookNode *cur = head; cur != NULL; cur = cur->next) { count++; }
+    return count;
 }
```

6. PHÂN TÍCH TRADE-OFF: CPPCHECK (TĨNH) VS. ADDRESSSANITIZER (ĐỘNG) TRONG CI/CD

6.1. Bảng so sánh toàn diện hai phương pháp

Tiêu chí so sánh	Phân tích Tĩnh (cppcheck - SAST)	Phân tích Động (AddressSanitizer - DAST)
Điều kiện thực thi	Chỉ cần mã nguồn thô (.c/.h), không cần biên dịch hay chạy chương trình.	Bắt buộc phải biên dịch thành mã máy và thực thi qua bộ test case thực tế.
Tốc độ thực thi	Rất nhanh (vài giây để quét toàn bộ kho mã nguồn lớn).	Chậm hơn đáng kể (tăng độ trễ CPU ~2x và bộ nhớ RAM ~3x khi chạy test).
Độ phủ mã (Coverage)	Bao quát 100% mọi dòng lệnh (kể cả các nhánh xử lý lỗi hiếm gặp).	Phụ thuộc 100% vào độ phủ của test case (nhánh nào không chạy thì không bắt được).
Báo động giả (False Positives)	Có thể xuất hiện báo động giả do giới hạn suy diễn của bộ phân tích cú pháp.	**Tuyệt đối không có báo động giả (Zero False Positives)** — hễ báo là lỗi thật 100%.
Lớp lỗi bắt tốt nhất	Biến chưa khởi tạo, lỗi cú pháp, dead code, con trỏ NULL hiển nhiên.	Tràn bộ đệm heap/stack thực tế, Use-After-Free, Double Free, Memory Leak.

6.2. Lập luận quyết định lựa chọn cho Pipeline CI/CD

CÂU HỎI: *"Nếu phải chọn CHỈ MỘT công cụ (cppcheck tĩnh hoặc AddressSanitizer động) để đưa vào pipeline CI/CD, bạn chọn công cụ nào? Vì sao?"*

LẬP LUẬN LỰA CHỌN: Nếu bắt buộc phải chọn duy nhất 1 công cụ cho pipeline tự động hóa CI/CD, lựa chọn tối ưu và an toàn nhất là **AddressSanitizer (ASan)**. Các lý do kỹ thuật mang tính quyết định bao gồm:

1. **Tính xác thực tuyệt đối (Zero False Positive):** Trong quy trình tự động hóa CI/CD, việc pipeline bị chặn (block build) bởi các báo động giả là nguyên nhân hàng đầu gây lãng phí thời gian của đội ngũ kỹ sư và làm suy giảm niềm tin vào hệ thống kiểm thử. Với ASan, mọi cảnh báo đều bắt nguồn từ một vi phạm vùng nhớ thực sự xảy ra trên RAM lúc chạy. Khi ASan báo lỗi, 100% đó là lỗi nghiêm trọng cần phải sửa.
2. **Khả năng ngăn chặn các lỗ hổng thảm họa:** Các lỗi bộ nhớ nguy hiểm nhất trong an toàn phần mềm (như Buffer Overflow dẫn đến thực thi mã từ xa RCE, hoặc Use-After-Free dẫn đến leo thang đặc quyền) đều diễn ra một cách động tại runtime thông qua tính toán con trỏ phức tạp. cppcheck hầu như bất lực trước các kịch bản này, trong khi ASan phát hiện và định vị chính xác vị trí lỗi.
3. **Dễ dàng tích hợp vào bộ kiểm thử đơn vị:** Mọi dự án phần mềm chuyên nghiệp đều đã có sẵn bộ kiểm thử tự động (Unit / Integration Tests). Việc kích hoạt thêm cờ `-fsanitize=address` vào quá trình chạy test không làm thay đổi luồng làm việc nhưng mang lại khả năng bảo vệ tối thượng.

KHUYẾN NGHỊ THỰC TẾ (MÔ HÌNH PHÒNG THỦ THEO CHIỀU SÂU - DEFENSE-IN-DEPTH):

Trong môi trường công nghiệp hiện đại, không bao giờ nên loại trừ một trong hai mà cần kết hợp cả hai theo mô hình *Shift-Left Security*: Sử dụng **cppcheck ở bước Pre-commit / Pull Request** để ngăn chặn nhanh các lỗi sơ đẳng trong 5 giây, và sử dụng **AddressSanitizer ở bước Test Execution / Nightly Build** để kiểm chứng chuyên sâu toàn bộ an toàn bộ nhớ.

7. KẾT QUẢ BỘ KIỂM THỬ TỰ ĐỘNG TOÀN DIỆN

Bộ kiểm thử tự động `tests/test_static_analysis.py` kiểm tra toàn diện 7 test case:

STT	Tên Test Case	Mục tiêu kiểm tra	Kết quả
1	test_buggy_compiles_and_runs	Xác nhận bản buggy biên dịch và chạy logic cơ bản	PASS
2	test_fixed_compiles_and_runs_correct_count	Xác nhận bản fixed biên dịch và đếm chính xác số lượng node	PASS
3	test_cppcheck_detects_issues_in_buggy_version	Xác nhận cppcheck phát hiện >= 2 lỗi bảo mật trên bản buggy	PASS
4	test_cppcheck_clean_on_fixed_version	Xác nhận bản fixed đạt 0 finding bảo mật	PASS
5	test_matches_baseline	Xác nhận kết quả phân tích khớp hoàn toàn với expected baseline	PASS
6	test_uninitialized_variable_garbage_value_impact	Kiểm tra phát hiện và bắt lỗi biến chưa khởi tạo (CWE-457)	PASS
7	test_memory_leak_node_orphan_fix	Kiểm tra xác nhận loại bỏ hoàn toàn các finding trên bản fixed	PASS

Lệnh thực thi: `python tests/test_static_analysis.py` --> Kết quả: **7/7 test da PASS (100%)**.

8. TỔNG KẾT BÀI THỰC HÀNH VÀ BÀI HỌC KINH NGHIỆM

1. Làm chủ kỹ thuật phân tích tĩnh mã nguồn bằng cppcheck, đọc hiểu cây AST/CFG và phân loại chính xác các mức độ nghiêm trọng của lỗi phần mềm.
2. Hiểu sâu sắc điểm mù của phân tích tĩnh đối với các bài toán con trỏ phức tạp (Pointer Aliasing), củng cố nhận thức về sự cần thiết của kiểm thử động.
3. Hoàn thiện kỹ năng lập trình cấu trúc dữ liệu linked list an toàn trong C, triệt tiêu hoàn toàn rò rỉ bộ nhớ và lỗi con trỏ NULL.

4. Xây dựng chiến lược kết hợp hài hòa giữa SAST và DAST trong đường ống CI/CD doanh nghiệp theo nguyên tắc phòng thủ theo chiều sâu.